Week10 作業：PCA 與 K-means 分群分析

學號 B1229009　廖晢勛　｜　資料：students_200.csv（200 位學生的 chinese、english 成績）

一、分析步驟與過程說明

1. **讀取與標準化**：讀入 students_200.csv，只取 chinese、english 兩欄（student_id、true_cluster 不參與分群），以 StandardScaler 標準化，使兩科在相同尺度上計算距離。
2. **原始資料分群**：對標準化後的原始 2 維資料使用 KMeans(n_clusters=3, n_init=10, random_state=42) 分群，計算 silhouette score。
3. **PCA 降維**：對標準化資料做 PCA，計算各主成分的特徵值、貢獻率與累積貢獻率，並畫出 Scree Plot。
4. **嘗試不同維度**：分別取 n_components = 1、2，各自重新用 KMeans(n=3) 分群並計算 silhouette score。
   ※ 題目建議可嘗試 1、2、3 維；但本資料只有 2 個特徵，PCA 的主成分數上限為 min(樣本數, 特徵數) = 2，因此無法降到 3 維，只比較 1 維與 2 維。
5. **比較並選出最適維度**：以 silhouette score 最高者為最佳降維維度。

二、使用 AI 工具協助的截圖

使用 AI 工具：Claude Code（Claude Pro），在終端機中下指令請 AI 撰寫並執行程式。

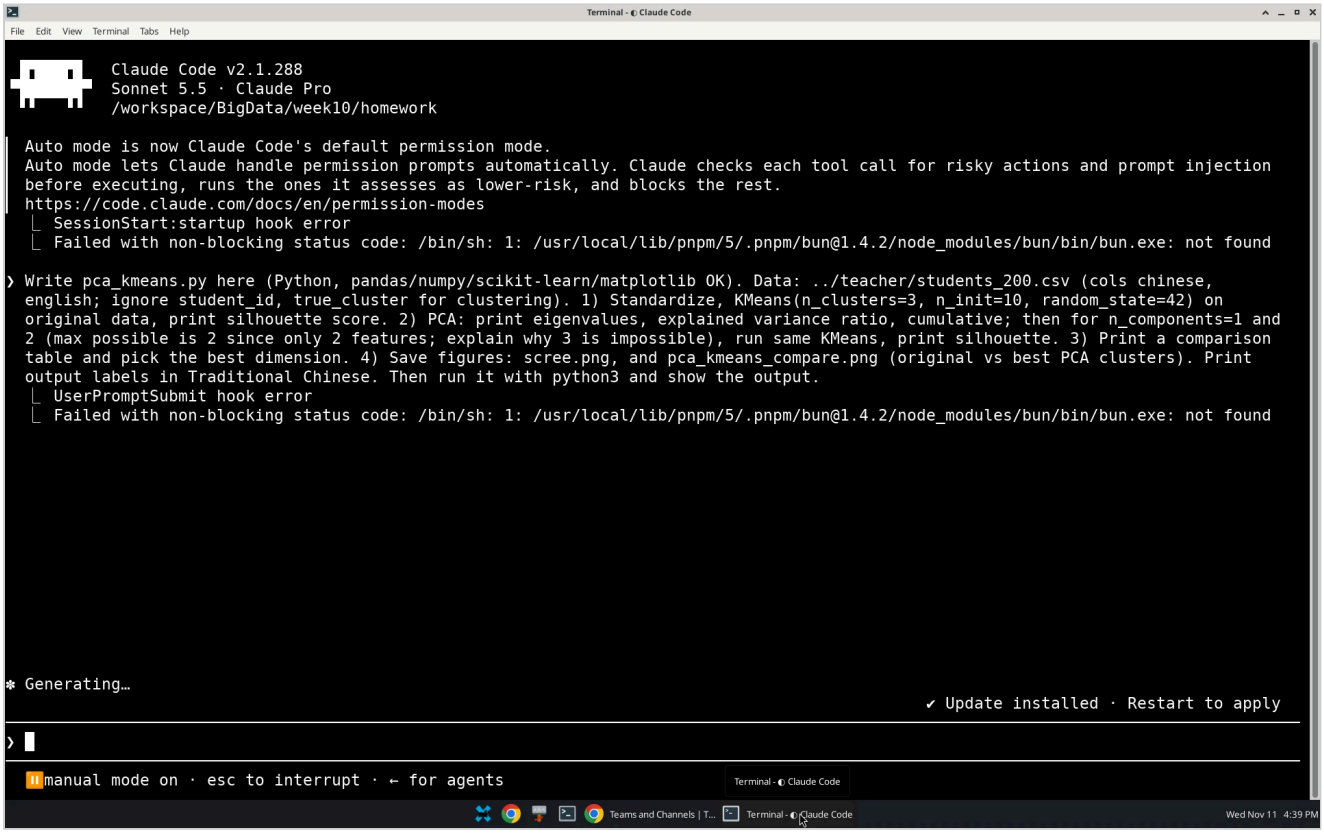


圖 1：向 Claude Code 輸入分析需求，AI 開始產生程式

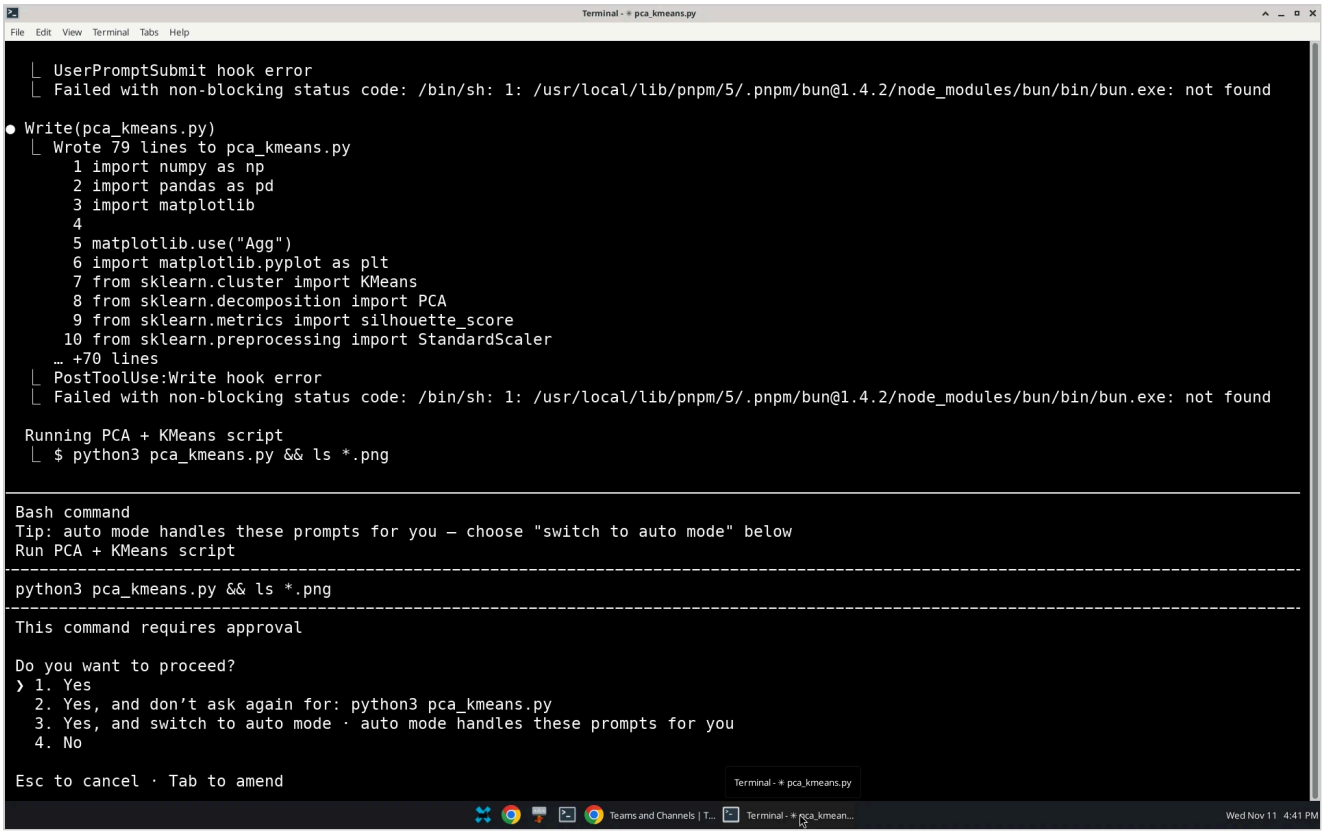


圖 2：AI 撰寫 pca_kmeans.py 並請求執行權限

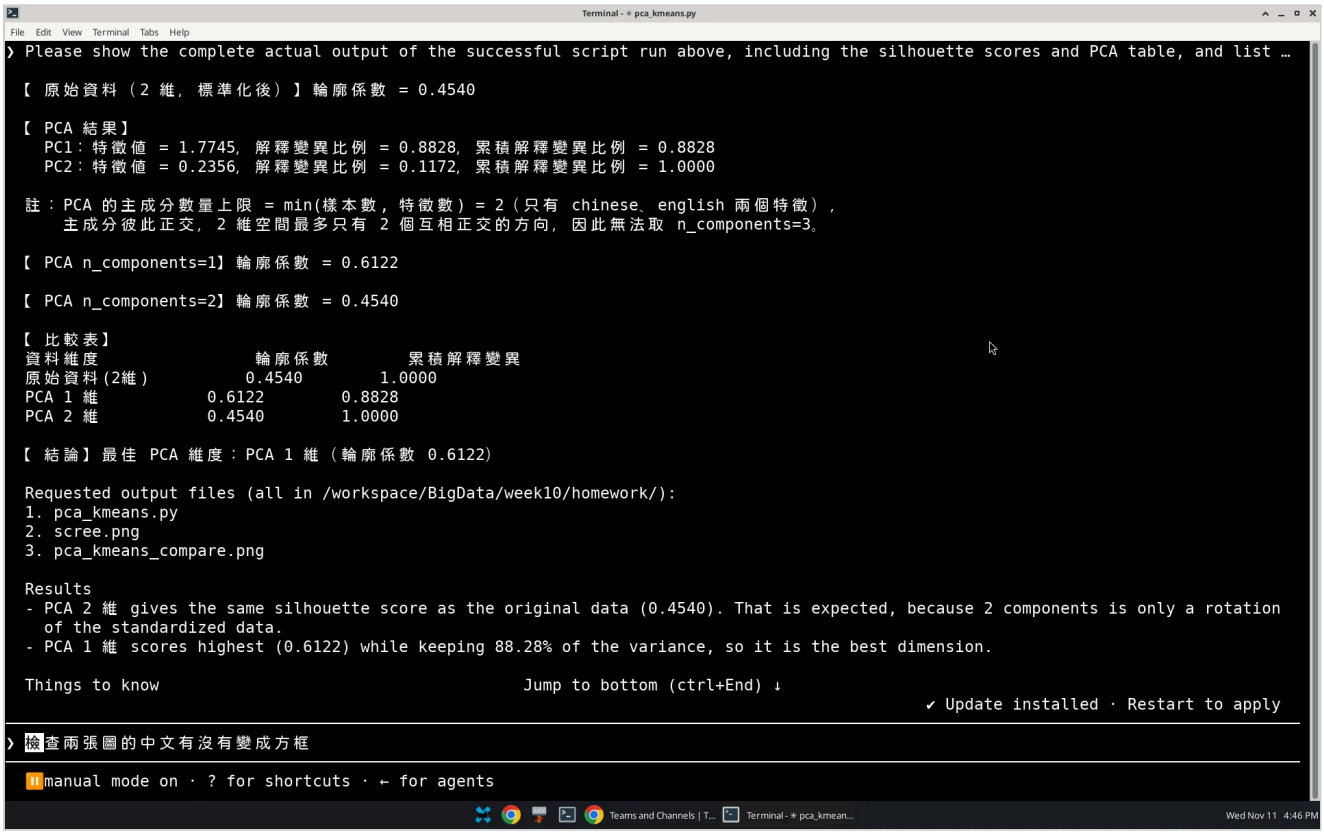


圖 3：AI 執行程式後的輸出（特徵值、貢獻率、silhouette score、比較表）

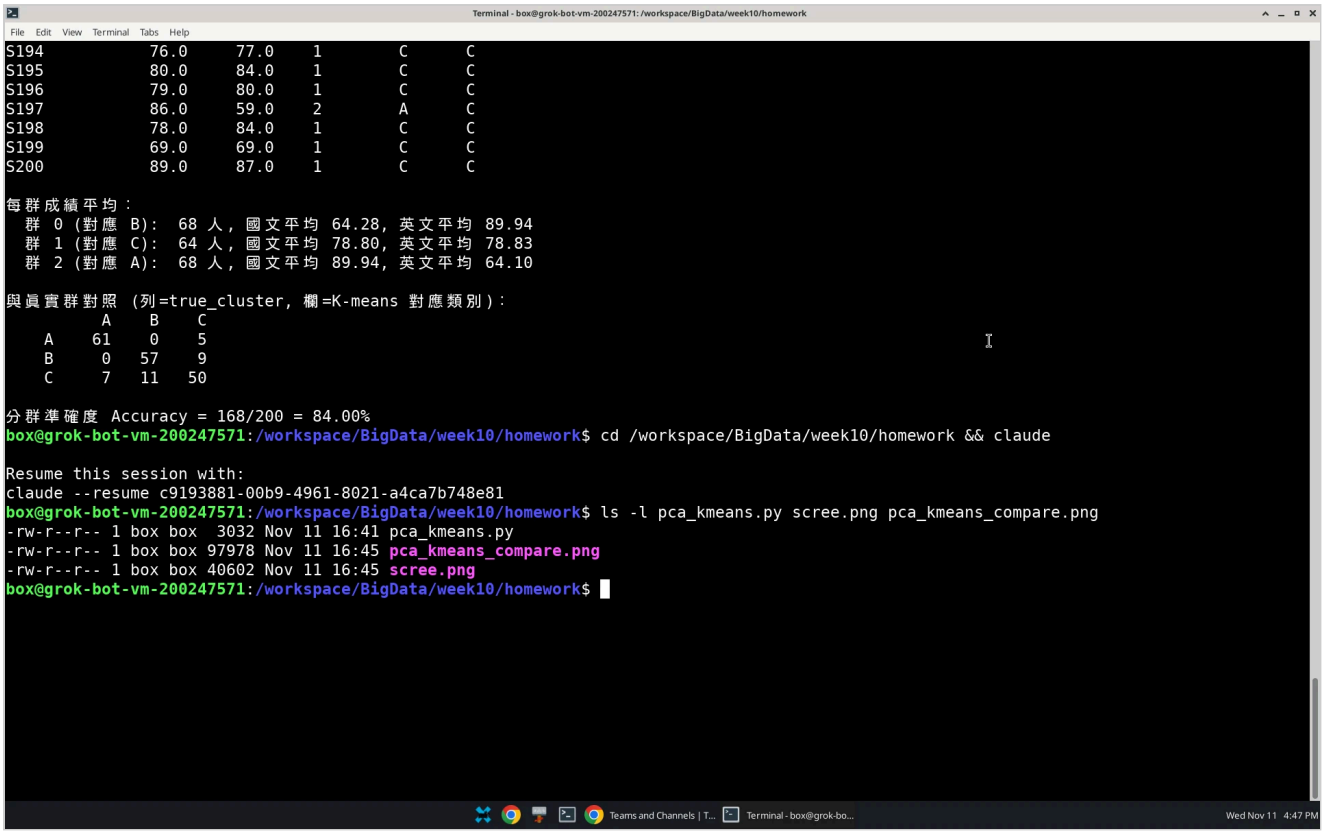

Terminal - box@grok-bot-vm-200247571: /workspace/BigData/week10/homework
File Edit View Terminal Tabs Help
S194 76.0 77.0 1 C C
S195 80.0 84.0 1 C C
S196 79.0 80.0 1 C C
S197 86.0 59.0 2 A C
S198 78.0 84.0 1 C C
S199 69.0 69.0 1 C C
S200 89.0 87.0 1 C C
每群成績平均：
群 0 (對應 B): 68 人，國文平均 64.28, 英文平均 89.94
群 1 (對應 C): 64 人，國文平均 78.80, 英文平均 78.83
群 2 (對應 A): 68 人，國文平均 89.94, 英文平均 64.10
與眞實群對照 (列=true_cluster, 欄=K-means 對應類別)：
A B C
A 61 0 5
B 0 57 9
C 7 11 50
分群準確度 Accuracy = 168/200 = 84.00%
box@grok-bot-vm-200247571:/workspace/BigData/week10/homework$ cd /workspace/BigData/week10/homework && claude
Resume this session with:
claude --resume c9193881-00b9-4961-8021-a4ca7b748e81
box@grok-bot-vm-200247571:/workspace/BigData/week10/homework$ ls -l pca_kmeans.py scree.png pca_kmeans_compare.png
-rw-r--r-- 1 box box 3032 Nov 11 16:41 pca_kmeans.py
-rw-r--r-- 1 box box 97978 Nov 11 16:45 pca_kmeans_compare.png
-rw-r--r-- 1 box box 40602 Nov 11 16:45 scree.png
box@grok-bot-vm-200247571:/workspace/BigData/week10/homework$
Teams and Channels | T...
Terminal - box@grok-bo...
Wed Nov 11 4:47 PM

圖 4：確認輸出檔案

三、程式碼（pca_kmeans.py）

```
import numpy as np
import pandas as pd
import matplotlib

matplotlib.use("Agg")
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA
from sklearn.metrics import silhouette_score
from sklearn.preprocessing import StandardScaler

plt.rcParams["font.sans-serif"] = ["Noto Sans CJK TC", "Noto Sans CJK JP", "Microsoft JhengHei", "PingFang TC",
"SimHei", "DejaVu Sans"]
plt.rcParams["axes.unicode_minus"] = False

df = pd.read_csv("../teacher/students_200.csv")
X = StandardScaler().fit_transform(df[["chinese", "english"]])


def run_kmeans(data):
    labels = KMeans(n_clusters=3, n_init=10, random_state=42).fit_predict(data)
    return labels, silhouette_score(data, labels)


# 1) 原始資料
labels0, s0 = run_kmeans(X)
print(f"【原始資料（2 維，標準化後)】輪廓係數 = {s0:.4f}")

# 2) PCA
pca = PCA().fit(X)
ev, evr = pca.explained_variance_, pca.explained_variance_ratio_
cum = np.cumsum(evr)
print("\n【PCA 結果】")
for i in range(len(ev)):
    print(f"  PC{i+1}：特徵值 = {ev[i]:.4f}，解釋變異比例 = {evr[i]:.4f}，累積解釋變異比例 = {cum[i]:.4f}")
print("\n註：PCA 的主成分數量上限 = min(樣本數, 特徵數) = 2（只有 chinese、english 兩個特徵)，")
print("    主成分彼此正交，2 維空間最多只有 2 個互相正交的方向，因此無法取 n_components=3。")

results = {"原始資料(2維)": (s0, X, labels0)}
for k in (1, 2):
    Z = PCA(n_components=k).fit_transform(X)
    lab, s = run_kmeans(Z)
    results[f"PCA {k} 維"] = (s, Z, lab)
    print(f"\n【PCA n_components={k}】輪廓係數 = {s:.4f}")

# 3) 比較表
print("\n【比較表】")
print(f"{'資料維度':<14}{'輪廓係數':>10}{'累積解釋變異':>14}")
cums = {"原始資料(2維)": 1.0, "PCA 1 維": cum[0], "PCA 2 維": cum[1]}
for name, (s, _, _) in results.items():
```

```
    print(f"{name:<14}{s:>10.4f}{cums[name]:>14.4f}")

pca_names = ["PCA 1 維", "PCA 2 維"]
best = max(pca_names, key=lambda n: results[n][0])
print(f"\n【結論】最佳 PCA 維度：{best}（輪廓係數 {results[best][0]:.4f}）")

# 4) 圖
plt.figure(figsize=(6, 4))
plt.plot([1, 2], ev, "o-")
plt.axhline(1, ls="--", c="gray", label="特徵值 = 1")
plt.xticks([1, 2])
plt.xlabel("主成分")
plt.ylabel("特徵值")
plt.title("陡坡圖 (Scree Plot)")
plt.legend()
plt.tight_layout()
plt.savefig("scree.png", dpi=150)

fig, ax = plt.subplots(1, 2, figsize=(11, 4.5))
for a, (name, xs) in zip(ax, [("原始資料(2維)", None), (best, None)]):
    s, Z, lab = results[name]
    # 1 維資料補 0 作為 y 軸以便繪圖
    y = Z[:, 1] if Z.shape[1] > 1 else np.zeros(len(Z))
    a.scatter(Z[:, 0], y, c=lab, cmap="viridis", s=18)
    a.set_title(f"{name}  輪廓係數 = {s:.4f}")
a0, a1 = ax
a0.set_xlabel("中文（標準化）"); a0.set_ylabel("英文（標準化）")
a1.set_xlabel("PC1"); a1.set_ylabel("PC2" if best == "PCA 2 維" else "（1 維，y 固定為 0）")
plt.tight_layout()
plt.savefig("pca_kmeans_compare.png", dpi=150)
```

四、輸出結果

```
【原始資料（2 維，標準化後)】輪廓係數 = 0.4540

【PCA 結果】
  PC1：特徵值 = 1.7745，解釋變異比例 = 0.8828，累積解釋變異比例 = 0.8828
  PC2：特徵值 = 0.2356，解釋變異比例 = 0.1172，累積解釋變異比例 = 1.0000

註：PCA 的主成分數量上限 = min(樣本數, 特徵數) = 2（只有 chinese、english 兩個特徵），
    主成分彼此正交，2 維空間最多只有 2 個互相正交的方向，因此無法取 n_components=3。

【PCA n_components=1】輪廓係數 = 0.6122

【PCA n_components=2】輪廓係數 = 0.4540

【比較表】
資料維度                  輪廓係數         累積解釋變異
原始資料(2維)           0.4540        1.0000
PCA 1 維           0.6122        0.8828
PCA 2 維           0.4540        1.0000

【結論】最佳 PCA 維度：PCA 1 維（輪廓係數 0.6122）
```

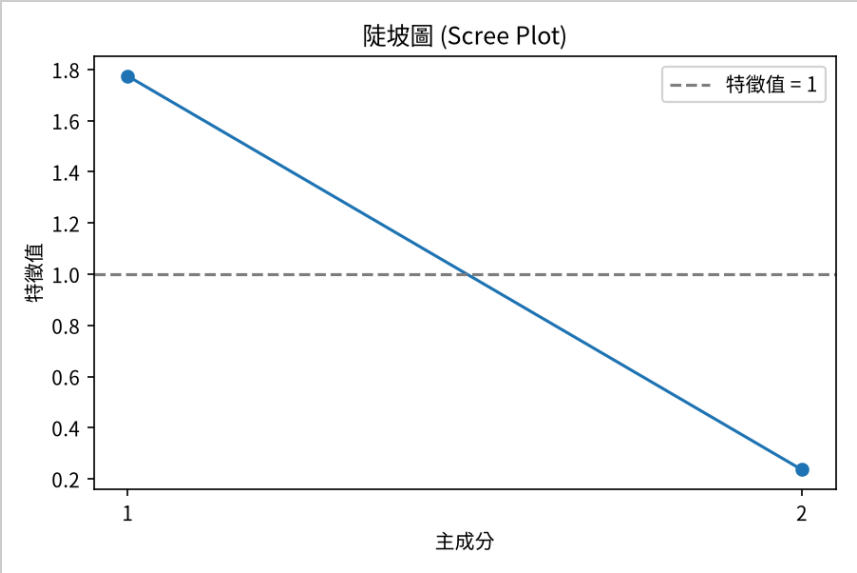


圖 5：Scree Plot（PC1 特徵值 1.7745，貢獻率 88.28%，已超過 80%）

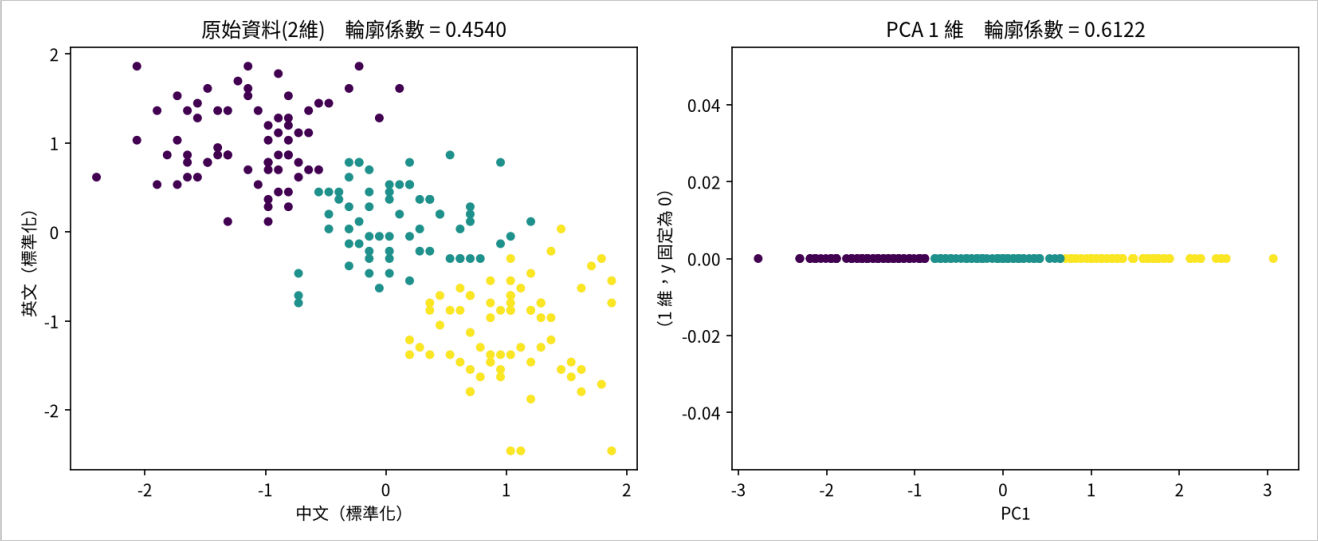


圖 6：原始資料（2 維）與 PCA 1 維的 K-means 分群結果

五、降維前後的 silhouette score 比較

資料維度	Silhouette score	累積解釋變異
原始資料（2 維，標準化）	0.4540	100.00%
PCA 1 維	**0.6122**	88.28%
PCA 2 維	0.4540	100.00%

結論：最適降維維度為 PCA 1 維（silhouette score 0.6122，高於原始資料的 0.4540）。

- PC1 一個主成分就保留了 88.28% 的資料變異，超過 80% 的門檻，足以代表主要資料結構。
- PCA 2 維只是把標準化後的資料做旋轉，點與點之間的距離不變，所以分群結果與 silhouette score 都和原始資料相同（0.4540）。
- 降到 1 維後，去掉了變異較小的 PC2 方向（多為雜訊），各群在 PC1 上分得更開、群內更緊密，因此 silhouette score 提高到 0.6122。